Automatic Logging with MLflow Tracking

Auto logging is a powerful feature that allows you to log metrics, parameters, and models without the need for explicit log statements. All you need to do is to call `mlflow.autolog()` before your training code.

```python
import mlflow

mlflow.autolog()

with mlflow.start_run():
    # your training code goes here
    ...
```

This will enable MLflow to automatically log various information about your run, including:

python

- **Parameters** - hyper params specified for the training, plus default values provided by the library if not explicitly set
- **Model Signature** - logs Model signature instance, which describes input and output schema of the model
- **Artifacts** - e.g. model checkpoints
- **Dataset** - dataset object used for training (if applicable), such as *tensorflow.data.Dataset*

How to Get started

Step 1 - Get MLflow

MLflow is available on PyPI. If you don't already have it installed on your system, you can install it with:

```bash
pip install mlflow
```

Step 2 - Insert `mlflow.autolog` in Your Code

For example, following code snippet shows how to enable autologging for a scikit-learn model:

```python
import mlflow

from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor

mlflow.autolog()

db = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(db.data, db.target)
```

Ask AI

```
rf = RandomForestRegressor(n_estimators=100, max_depth=6, max_features=3)
# MLflow triggers logging automatically upon model fitting
rf.fit(X_train, y_train)
```

Step 3 - Execute Your Code

```bash
python YOUR_ML_CODE.py
```

Step 4 - View Your Results in the MLflow UI

Once your training job finishes, you can run following command to launch the MLflow UI:

```bash
mlflow server --port 8080
```

Then, navigate to `http://localhost:8080` in your browser to view the results.

Customize Autologging Behavior

You can also control the behavior of autologging by passing arguments to `mlflow.autolog()` function. For example, you can disable logging of model checkpoints and associate tags with your run as follows:

```python
import mlflow

mlflow.autolog(
    log_model_signatures=False,
    extra_tags={"YOUR_TAG": "VALUE"},
)
```

See `mlflow.autolog()` for the full set of arguments you can use.

Enable / Disable Autologging for Specific Libraries

One common use case is to enable/disable autologging for a specific library. For example, if you train your model on PyTorch but use scikit-learn for data preprocessing, you may want to disable autologging for scikit-learn while keeping it enabled for PyTorch. You can achieve this by either (1) enable autologging only for PyTorch using PyTorch flavor (2) disable autologging for scikit-learn using its flavor with `disable=True`.

```python
import mlflow

# Option 1: Enable autologging only for PyTorch
mlflow.pytorch.autolog()

# Option 2: Disable autologging for scikit-learn, but enable it for other libraries
mlflow.sklearn.autolog(disable=True)
mlflow.autolog()
```

Ask AI

Supported Libraries

> **NOTE**
> The generic autolog function `mlflow.autolog()` enables autologging for each supported library you have installed as soon as you import it. Alternatively, you can use library-specific autolog calls such as `mlflow.pytorch.autolog()` to explicitly enable (or disable) autologging for a particular library.

The following list covers the most popular libraries that support autologging within MLflow:

> **NOTE**
> There are many more integrations that support autologging and the list of supported libraries is constantly growing. See the dedicated pages for further guidance on whether autologging support is available for a given library.

For flavors that automatically save models as an artifact, additional files for dependency management are logged.

Keras/TensorFlow

Call the generic autolog function or `mlflow.tensorflow.autolog()` before your training code to enable automatic logging of metrics and parameters. As an example, try running the Keras/Tensorflow example.

Note that only versions of `tensorflow>=2.3` are supported. The respective metrics associated with `tf.estimator` and `EarlyStopping` are automatically logged. As an example, try running the Keras/TensorFlow example.

Autologging captures the following information:

Framework	Metrics	Parameters	Tags	Artifacts
`tf.keras`	Training loss; validation loss; user-specified metrics	`fit()` parameters; optimizer name; learning rate; epsilon	--	Model summary on training start; MLflow Model (Keras model); TensorBoard logs on training end
`tf.keras.callbacks.EarlyStopping`	Metrics from the `EarlyStopping` callbacks. For example, `stopped_epoch`, `restored_epoch`, `restore_best_weight`, etc	`fit()` parameters from `EarlyStopping`. For example, `min_delta`, `patience`, `baseline`, `restore_best_weights`, etc	--	--

If no active run exists when `autolog()` captures data, MLflow will automatically create a run to log information to. Also, MLflow will then automatically end the run once training ends via calls to `tf.keras.fit()`.

If a run already exists when `autolog()` captures data, MLflow will log to that run but not automatically end that run after training. You will have to manually stop the run if you wish to start a new run context for logging to a new run.

LightGBM

Ask AI

Call the generic autolog function `mlflow.lightgbm.autolog()` before your training code to enable automatic logging of metrics and parameters.

Autologging captures the following information:

Framework	Metrics	Parameters	Tags	Artifacts
LightGBM	user-specified metrics	lightgbm.train parameters	--	MLflow Model (LightGBM model) with model signature on training end; feature importance; input example;

If early stopping is activated, metrics at the best iteration will be logged as an extra step/iteration.

Paddle

Call the generic autolog function `mlflow.paddle.autolog()` before your training code to enable automatic logging of metrics and parameters.

Autologging captures the following information:

Framework	Metrics	Parameters	Tags	Artifacts
Paddle	user-specified metrics	paddle.Model.fit parameters	--	MLflow Model (Paddle model) with model signature on training end

PySpark

Call `mlflow.pyspark.ml.autolog()` before your training code to enable automatic logging of metrics, params, and models. See example usage with PySpark.

Autologging for pyspark ml estimators captures the following information:

Metrics	Parameters	Tags	Artifacts
Post training metrics obtained by `Evaluator.evaluate`	Parameters obtained by `Estimator.fit`	• Class name • Fully qualified class name	• MLflow Model containing a fitted estimator • `metric_info.json` for post training metrics

PyTorch

Call the generic autolog function `mlflow.pytorch.autolog()` before your PyTorch Lightning training code to enable automatic logging of metrics, parameters, and models. See example usages here. Note that currently, PyTorch autologging supports only models trained using PyTorch Lightning.

Autologging is triggered on calls to `pytorch_lightning.trainer.Trainer.fit` and captures the following information:

Framework/module	Metrics	Parameters	Tags	Artifacts
`pytorch_lightning.trainer.Trainer`	Training loss; validation loss; average_test_accuracy; user-defined-metrics	`fit()` parameters; optimizer name; learning rate; epsilon.	--	Model summary ning MLflow Model (PyTorch

Ask AI

Framework/module	Metrics	Parameters	Tags	Artifacts
				model) on training end;
`pytorch_lightning.callbacks.earlystopping`	Training loss; validation loss; average_test_accuracy; user-defined-metrics. Metrics from the `EarlyStopping` callbacks. For example, `spotted_epoch`, `restored_epoch`, `restore_best_weight`, etc	`fit()` parameters; optimizer name; learning rate; epsilon. Parameters from the `EarlyStopping` callbacks. For example, `min_delta`, `patience`, `baseline`, `restore_best_weights`, etc	--	Model summary on training start; MLflow Model (PyTorch model) on training end; Best PyTorch model checkpoint, if training stops due to early stopping callback.

If no active run exists when `autolog()` captures data, MLflow will automatically create a run to log information, ending the run once the call to `pytorch_lightning.trainer.Trainer.fit()` completes.

If a run already exists when `autolog()` captures data, MLflow will log to that run but not automatically end that run after training.

> **NOTE**
> - Parameters not explicitly passed by users (parameters that use default values) while using `pytorch_lightning.trainer.Trainer.fit()` are not currently automatically logged
> - In case of a multi-optimizer scenario (such as usage of autoencoder), only the parameters for the first optimizer are logged

Scikit-learn

Call `mlflow.sklearn.autolog()` before your training code to enable automatic logging of sklearn metrics, params, and models. See example usage here.

Autologging for estimators (e.g. LinearRegression) and meta estimators (e.g. Pipeline) creates a single run and logs:

Metrics	Parameters	Tags	Artifacts
Training score obtained by `estimator.score`	Parameters obtained by `estimator.get_params`	• Class name • Fully qualified class name	Fitted estimator

Autologging for parameter search estimators (e.g. GridSearchCV) creates a single parent run and ne

Ask AI

text

```
- Parent run
  - Child run 1
  - Child run 2
  - ...
```

containing the following data:

Run type	Metrics	Parameters	Tags	Artifacts
Parent	Training score	• Parameter search estimator's parameters • Best parameter combination	• Class name • Fully qualified class name	• Fitted parameter search estimator • Fitted best estimator • Search results csv file
Child	CV test score for each parameter combination	Each parameter combination	• Class name • Fully qualified class name	--

Spark

Initialize a SparkSession with the mlflow-spark JAR attached (e.g. `SparkSession.builder.config("spark.jars.packages", "org.mlflow.mlflow-spark")`) and then call the generic autolog function `mlflow.spark.autolog()` to enable automatic logging of Spark datasource information at read-time, without the need for explicit log statements. Note that autologging of Spark ML (MLlib) models is not yet supported.

Autologging captures the following information:

Framework	Metrics	Parameters	Tags	Artifacts
Spark	--	--	Single tag containing source path, version, format. The tag contains one line per datasource	--

NOTE

- Moreover, Spark datasource autologging occurs asynchronously - as such, it's possible (though unlikely) to see race conditions when launching short-lived MLflow runs that result in datasource information not being logged.

IMPORTANT

With Pyspark 3.2.0 or above, Spark datasource autologging requires `PYSPARK_PIN_THREAD` environment variable to be set to `false`.

Statsmodels

Call the generic autolog function `mlflow.statsmodels.autolog()` before your training code to enab of metrics and parameters.

Ask AI

Autologging captures the following information:

Framework	Metrics	Parameters	Tags	Artifacts
Statsmodels	user-specified metrics	statsmodels.base.model.Model.fit parameters	--	MLflow Model (statsmodels.base.wrapper.ResultsWrapper) on training end

NOTE

- Each model subclass that overrides *fit* expects and logs its own parameters.

XGBoost

Call the generic autolog function `mlflow.xgboost.autolog()` before your training code to enable automatic logging of metrics and parameters.

Autologging captures the following information:

Framework	Metrics	Parameters	Tags	Artifacts
XGBoost	user-specified metrics	xgboost.train parameters	--	MLflow Model (XGBoost model) with model signature on training end; feature importance; input example

If early stopping is activated, metrics at the best iteration will be logged as an extra step/iteration.

© 2025 MLflow Project, a Series of LF Projects, LLC.

Ask AI